A Beginner's Guide to Clean Dockerfiles

Writing a Dockerfile is easy, but writing a good one saves storage space and keeps your applications secure. These foundational practices will help you build smaller, faster containers right from day one.

What you'll learn:

1. Start with Lightweight Base Images
2. Master the Build Cache
3. Keep Your Image Diets Strict
4. Lock Down Container Security

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Start with Lightweight Base Images

When you write a Dockerfile, your very first line is the FROM instruction. Choosing a slim base image like Alpine Linux drastically reduces your final download size and removes unnecessary software tools that hackers could exploit. Always pick the smallest official image that still supports your application runtime.

```
FROM python:3.9-slim
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Master the Build Cache

Docker reads your Dockerfile line by line and saves the result of each step as a layer. If a line hasn't changed, Docker reuses the saved layer instead of doing the work again. Place files that change rarely—like dependency manifest files—before your actual source code to make your builds blazing fast.

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Keep Your Image Diets Strict

Every time you install a package manager tool or download temporary files, they get baked permanently into that Docker layer. To keep your images lean, always install packages and clean up their cache directories inside a single RUN command using the && operator.

```
RUN apt-get update && apt-get install -y git && rm -rf /var/lib/apt/lists/*
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Lock Down Container Security

By default, Docker containers run your application as the root user, which is a major security risk if your app gets compromised. Adding a simple non-root user instruction ensures your application operates with limited privileges, keeping your underlying server safe.

```
RUN useradd -m myuser
USER myuser
CMD ["python", "app.py"]
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always use a .dockerignore file to exclude local logs and dependencies
+ Pin your base image versions (e.g., node:18.16.0 instead of node:latest)
+ Combine related RUN commands to minimize the number of image layers
+ Test your container locally before pushing it to any remote registry
+ Use multi-stage builds when compiling heavy compiled languages like Go or Java

Watch Out For

! Hardcoding secret API keys or passwords directly inside the Dockerfile
! Forgetting to clean up package manager caches which bloat image sizes
! Running your web application container as the root user
! Copying your entire local project directory before installing dependencies

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh